

Documentation Index

Fetch the complete documentation index at: <https://code.claude.com/docs/llms.txt>

Use this file to discover all available pages before exploring further.

How Claude remembers your project

*Give Claude persistent instructions with **CLAUDE.md** files, and let Claude accumulate learnings automatically with auto memory.*

Each Claude Code session begins with a fresh context window. Two mechanisms carry knowledge across sessions:

- **CLAUDE.md files:** instructions you write to give Claude persistent context
- **Auto memory:** notes Claude writes itself based on your corrections and preferences

This page covers how to:

- [Write and organize CLAUDE.md files](#)
- [Scope rules to specific file types](#) with `.claude/rules/`
- [Configure auto memory](#) so Claude takes notes automatically
- [Troubleshoot](#) when instructions aren't being followed

CLAUDE.md vs auto memory

Claude Code has two complementary memory systems. Both are loaded at the start of every conversation. Claude treats them as context, not enforced configuration. The more specific and concise your instructions, the more consistently Claude follows them.

	CLAUDE.MD FILES	AUTO MEMORY
Who writes it	You	Claude
What it contains	Instructions and rules	Learnings and patterns
Scope	Project, user, or org	Per working tree
Loaded into	Every session	Every session (first 200 lines or 25KB)
Use for	Coding standards, workflows, project architecture	Build commands, debugging insights, preferences Claude discovers

Use CLAUDE.md files when you want to guide Claude's behavior. Auto memory lets Claude learn from your corrections without manual effort.

Subagents can also maintain their own auto memory. See [subagent configuration](#) for details.

CLAUDE.md files

CLAUDE.md files are markdown files that give Claude persistent instructions for a project, your personal workflow, or your entire organization. You write these files in plain text; Claude reads them at the start of every session.

Choose where to put CLAUDE.md files

CLAUDE.md files can live in several locations, each with a different scope. More specific locations take precedence over broader ones.

SCOPE	LOCATION	PURPOSE	USE CASE EXAMPLES	SHARED WITH
Managed policy	• macOS: <code>/Library/Application Support/ClaudeCode/CLAUDE.md</code> • Linux and WSL: <code>/etc/claude-code/CLAUDE.md</code> • Windows: <code>C:\Program Files\ClaudeCode\CLAUDE.md</code>	Organization-wide instructions managed by IT/DevOps	Company coding standards, security policies, compliance requirements	All users in organization
Project instructions	<code>./CLAUDE.md</code> OR <code>./.claude/CLAUDE.md</code>	Team-shared instructions for the project	Project architecture, coding standards, common workflows	Team members via source control
User instructions	<code>~/.claude/CLAUDE.md</code>	Personal preferences for all projects	Code styling preferences, personal tooling shortcuts	Just you (all projects)

CLAUDE.md files in the directory hierarchy above the working directory are loaded in full at launch. CLAUDE.md files in subdirectories load on demand when Claude reads files in those directories. See [How CLAUDE.md files load](#) for the full resolution order.

For large projects, you can break instructions into topic-specific files using [project rules](#). Rules let you scope instructions to specific file types or subdirectories.

Set up a project CLAUDE.md

A project CLAUDE.md can be stored in either `./CLAUDE.md` or `./.claude/CLAUDE.md`. Create this file and add instructions that apply to anyone working on the project: build and test commands, coding standards, architectural decisions, naming conventions, and common workflows. These instructions are shared with your team through version control, so focus on project-level standards rather than personal preferences.

Run `/init` to generate a starting CLAUDE.md automatically. Claude analyzes your codebase and creates a file with build commands, test instructions, and project conventions it discovers. If a CLAUDE.md already exists, `/init` suggests improvements rather than overwriting it. Refine from there with instructions Claude wouldn't discover on its own.

Set `CLAUDE_CODE_NEW_INIT=true` to enable an interactive multi-phase flow. `/init` asks which artifacts to set up: CLAUDE.md files, skills, and hooks. It then explores your codebase with a subagent, fills in gaps via follow-up questions, and presents a reviewable proposal before writing any files.

Write effective instructions

CLAUDE.md files are loaded into the context window at the start of every session, consuming tokens alongside your conversation. The [context window visualization](#) shows where CLAUDE.md loads relative to the rest of the startup context. Because they're context rather than enforced configuration, how you write instructions affects how reliably Claude follows them. Specific, concise, well-structured instructions work best.

Size: target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence. If your instructions are growing large, split them using [imports](#) or `.claude/rules/` files.

Structure: use markdown headers and bullets to group related instructions. Claude scans structure the same way readers do: organized sections are easier to follow than dense paragraphs.

Specificity: write instructions that are concrete enough to verify. For example:

- "Use 2-space indentation" instead of "Format code properly"
- "Run `npm test` before committing" instead of "Test your changes"
- "API handlers live in `src/api/handlers/`" instead of "Keep files organized"

Consistency: if two rules contradict each other, Claude may pick one arbitrarily. Review your CLAUDE.md files, nested CLAUDE.md files in subdirectories, and `.claude/rules/` periodically to remove outdated or conflicting instructions. In monorepos, use `claudeMdExcludes` to skip CLAUDE.md files from other teams that aren't relevant to your work.

Import additional files

CLAUDE.md files can import additional files using `@path/to/import` syntax. Imported files are expanded and loaded into context at launch alongside the CLAUDE.md that references them.

Both relative and absolute paths are allowed. Relative paths resolve relative to the file containing the import, not the working directory. Imported files can recursively import other files, with a maximum depth of five hops.

To pull in a README, package.json, and a workflow guide, reference them with `@` syntax anywhere in your CLAUDE.md:

```
See @README for project overview and @package.json for available
npm commands for this project.

# Additional Instructions
- git workflow @docs/git-instructions.md
```

For personal preferences you don't want to check in, import a file from your home directory. The import goes in the shared CLAUDE.md, but the file it points to stays on your machine:

```
# Individual Preferences
- @~/ .claude/my-project-instructions.md
```

The first time Claude Code encounters external imports in a project, it shows an approval dialog listing the files. If you decline, the imports stay disabled and the dialog does not appear again.

For a more structured approach to organizing instructions, see [.claude/rules/](#).

AGENTS.md

Claude Code reads `CLAUDE.md`, not `AGENTS.md`. If your repository already uses `AGENTS.md` for other coding agents, create a `CLAUDE.md` that imports it so both tools read the same instructions without duplicating them. You can also add Claude-specific instructions below the import. Claude loads the imported file at session start, then appends the rest:

```
@AGENTS.md

## Claude Code

Use plan mode for changes under `src/billing/`.
```

How CLAUDE.md files load

Claude Code reads CLAUDE.md files by walking up the directory tree from your current working directory, checking each directory along the way. This means if you run Claude Code in `foo/bar/`, it loads instructions from both `foo/bar/CLAUDE.md` and `foo/CLAUDE.md`.

Claude also discovers CLAUDE.md files in subdirectories under your current working directory. Instead of loading them at launch, they are included when Claude reads files in those subdirectories.

If you work in a large monorepo where other teams' CLAUDE.md files get picked up, use `claudeMdExcludes` to skip them.

Block-level HTML comments (`<!-- maintainer notes -->`) in CLAUDE.md files are stripped before the content is injected into Claude's context. Use them to leave notes for human maintainers without spending context tokens on them. Comments inside code blocks are preserved. When you open a CLAUDE.md file directly with the Read tool, comments remain visible.

Load from additional directories

The `--add-dir` flag gives Claude access to additional directories outside your main working directory. By default, CLAUDE.md files from these directories are not loaded.

To also load CLAUDE.md files from additional directories, including `CLAUDE.md`, `.claude/CLAUDE.md`, and `.claude/rules/*.md`, set the `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` environment variable:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add-dir
../shared-config
```

Organize rules with `.claude/rules/`

For larger projects, you can organize instructions into multiple files using the `.claude/rules/` directory. This keeps instructions modular and easier for teams to maintain. Rules can also be [scoped to specific file paths](#), so they only load into context when Claude works with matching files, reducing noise and saving context space.

Rules load into context every session or when matching files are opened. For task-specific instructions that don't need to be in context all the time, use [skills](#) instead, which only load when you invoke them or when Claude determines they're relevant to your prompt.

Set up rules

Place markdown files in your project's `.claude/rules/` directory. Each file should cover one topic, with a descriptive filename like `testing.md` or `api-design.md`. All `.md` files are discovered recursively, so you can organize rules into subdirectories like `frontend/` or `backend/`:

```
your-project/
├── .claude/
│   ├── CLAUDE.md          # Main project instructions
│   └── rules/
│       ├── code-style.md  # Code style guidelines
│       ├── testing.md     # Testing conventions
│       └── security.md    # Security requirements
```

Rules without [paths](#) [frontmatter](#) are loaded at launch with the same priority as `.claude/CLAUDE.md`.

Path-specific rules

Rules can be scoped to specific files using YAML frontmatter with the `paths` field. These conditional rules only apply when Claude is working with files matching the specified patterns.

```
---
paths:
  - "src/api/**/*.ts"
---

# API Development Rules

- All API endpoints must include input validation
- Use the standard error response format
- Include OpenAPI documentation comments
```

Rules without a `paths` field are loaded unconditionally and apply to all files. Path-scoped rules trigger when Claude reads files matching the pattern, not on every tool use.

Use glob patterns in the `paths` field to match files by extension, directory, or any combination:

PATTERN	MATCHES
<code>**/*.ts</code>	All TypeScript files in any directory
<code>src/**/*.ts</code>	All files under <code>src/</code> directory
<code>*.md</code>	Markdown files in the project root
<code>src/components/*.tsx</code>	React components in a specific directory

You can specify multiple patterns and use brace expansion to match multiple extensions in one pattern:

```
---
paths:
  - "src/**/*.{ts,tsx}"
  - "lib/**/*.ts"
  - "tests/**/*.test.ts"
---
```

Share rules across projects with symlinks

The `.claude/rules/` directory supports symlinks, so you can maintain a shared set of rules and link them into multiple projects. Symlinks are resolved and loaded normally, and circular symlinks are detected and handled gracefully.

This example links both a shared directory and an individual file:

```
ln -s ~/shared-claude-rules .claude/rules/shared
ln -s ~/company-standards/security.md .claude/rules/security.md
```

User-level rules

Personal rules in `~/.claude/rules/` apply to every project on your machine. Use them for preferences that aren't project-specific:


```
~/.claude/rules/  
├─ preferences.md    # Your personal coding preferences  
└─ workflows.md     # Your preferred workflows
```

User-level rules are loaded before project rules, giving project rules higher priority.

Manage CLAUDE.md for large teams

For organizations deploying Claude Code across teams, you can centralize instructions and control which CLAUDE.md files are loaded.

Deploy organization-wide CLAUDE.md

Organizations can deploy a centrally managed CLAUDE.md that applies to all users on a machine. This file cannot be excluded by individual settings.

- * macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`
- * Linux and WSL: `/etc/claude-code/CLAUDE.md`
- * Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`

Use MDM, Group Policy, Ansible, or similar tools to distribute the file across developer machines. See [managed settings](#) for other organization-wide configuration options.

A managed CLAUDE.md and [managed settings](#) serve different purposes. Use settings for technical enforcement and CLAUDE.md for behavioral guidance:

CONCERN	CONFIGURE IN
Block specific tools, commands, or file paths	Managed settings: <code>permissions.deny</code>
Enforce sandbox isolation	Managed settings: <code>sandbox.enabled</code>
Environment variables and API provider routing	Managed settings: <code>env</code>
Authentication method and organization lock	Managed settings: <code>forceLoginMethod</code> , <code>forceLoginOrgUUID</code>
Code style and quality guidelines	Managed CLAUDE.md
Data handling and compliance reminders	Managed CLAUDE.md
Behavioral instructions for Claude	Managed CLAUDE.md

Settings rules are enforced by the client regardless of what Claude decides to do. CLAUDE.md instructions shape Claude's behavior but are not a hard enforcement layer.

Exclude specific CLAUDE.md files

In large monorepos, ancestor CLAUDE.md files may contain instructions that aren't relevant to your work. The `claudeMdExcludes` setting lets you skip specific files by path or glob pattern.

This example excludes a top-level CLAUDE.md and a rules directory from a parent folder. Add it to `.claude/settings.local.json` so the exclusion stays local to your machine:

```
{
  "claudeMdExcludes": [
    "**/monorepo/CLAUDE.md",
    "/home/user/monorepo/other-team/.claude/rules/**"
  ]
}
```

Patterns are matched against absolute file paths using glob syntax. You can configure `claudeMdExcludes` at any [settings layer](#): user, project, local, or managed policy. Arrays merge across layers.

Managed policy CLAUDE.md files cannot be excluded. This ensures organization-wide instructions always apply regardless of individual settings.

Auto memory

Auto memory lets Claude accumulate knowledge across sessions without you writing anything. Claude saves notes for itself as it works: build commands, debugging insights, architecture notes, code style preferences, and workflow habits. Claude doesn't save something every session. It decides what's worth remembering based on whether the information would be useful in a future conversation.

Auto memory requires Claude Code v2.1.59 or later. Check your version with `claude --version`.

Enable or disable auto memory

Auto memory is on by default. To toggle it, open `/memory` in a session and use the auto memory toggle, or set `autoMemoryEnabled` in your project settings:

```
{
  "autoMemoryEnabled": false
}
```

To disable auto memory via environment variable, set `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`.

Storage location

Each project gets its own memory directory at `~/.claude/projects/<project>/memory/`. The `<project>` path is derived from the git repository, so all worktrees and subdirectories within the same repo share one auto memory directory. Outside a git repo, the project root is used instead.

To store auto memory in a different location, set `autoMemoryDirectory` in your user or local settings:

```
{
  "autoMemoryDirectory": "~/my-custom-memory-dir"
}
```

This setting is accepted from policy, local, and user settings. It is not accepted from project settings (`.claude/settings.json`) to prevent a shared project from redirecting auto memory writes to sensitive locations.

The directory contains a `MEMORY.md` entrypoint and optional topic files:

```
~/.claude/projects/<project>/memory/  
├─ MEMORY.md           # Concise index, loaded into every session  
├─ debugging.md        # Detailed notes on debugging patterns  
├─ api-conventions.md  # API design decisions  
└─ ...                 # Any other topic files Claude creates
```

`MEMORY.md` acts as an index of the memory directory. Claude reads and writes files in this directory throughout your session, using `MEMORY.md` to keep track of what's stored where.

Auto memory is machine-local. All worktrees and subdirectories within the same git repository share one auto memory directory. Files are not shared across machines or cloud environments.

How it works

The first 200 lines of `MEMORY.md`, or the first 25KB, whichever comes first, are loaded at the start of every conversation. Content beyond that threshold is not loaded at session start. Claude keeps `MEMORY.md` concise by moving detailed notes into separate topic files.

This limit applies only to `MEMORY.md`. `CLAUDE.md` files are loaded in full regardless of length, though shorter files produce better adherence.

Topic files like `debugging.md` or `patterns.md` are not loaded at startup. Claude reads them on demand using its standard file tools when it needs the information.

Claude reads and writes memory files during your session. When you see "Writing memory" or "Recalled memory" in the Claude Code interface, Claude is actively updating or reading from `~/.claude/projects/<project>/memory/`.

Audit and edit your memory

Auto memory files are plain markdown you can edit or delete at any time. Run `/memory` to browse and open memory files from within a session.

View and edit with `/memory`

The `/memory` command lists all CLAUDE.md and rules files loaded in your current session, lets you toggle auto memory on or off, and provides a link to open the auto memory folder. Select any file to open it in your editor.

When you ask Claude to remember something, like "always use pnpm, not npm" or "remember that the API tests require a local Redis instance," Claude saves it to auto memory. To add instructions to CLAUDE.md instead, ask Claude directly, like "add this to CLAUDE.md," or edit the file yourself via `/memory`.

Troubleshoot memory issues

These are the most common issues with CLAUDE.md and auto memory, along with steps to debug them.

Claude isn't following my CLAUDE.md

CLAUDE.md content is delivered as a user message after the system prompt, not as part of the system prompt itself. Claude reads it and tries to follow it, but there's no guarantee of strict compliance, especially for vague or conflicting instructions.

To debug:

- Run `/memory` to verify your CLAUDE.md files are being loaded. If a file isn't listed, Claude can't see it.
- Check that the relevant CLAUDE.md is in a location that gets loaded for your session (see [Choose where to put CLAUDE.md files](#)).
- Make instructions more specific. "Use 2-space indentation" works better than "format code nicely."
- Look for conflicting instructions across CLAUDE.md files. If two files give different guidance for the same behavior, Claude may pick one arbitrarily.

For instructions you want at the system prompt level, use `--append-system-prompt`. This must be passed every invocation, so it's better suited to scripts and automation than interactive use.

Use the `InstructionsLoaded` hook to log exactly which instruction files are loaded, when they load, and why. This is useful for debugging path-specific rules or lazy-loaded files in subdirectories.

I don't know what auto memory saved

Run `/memory` and select the auto memory folder to browse what Claude has saved. Everything is plain markdown you can read, edit, or delete.

My CLAUDE.md is too large

Files over 200 lines consume more context and may reduce adherence. Move detailed content into separate files referenced with `@path` imports (see [Import additional files](#)), or split your instructions across `.claude/rules/` files.

Instructions seem lost after `/compact`

CLAUDE.md fully survives compaction. After `/compact`, Claude re-reads your CLAUDE.md from disk and re-injects it fresh into the session. If an instruction disappeared after compaction, it was given only in conversation, not written to CLAUDE.md. Add it to CLAUDE.md to make it persist across sessions.

See [Write effective instructions](#) for guidance on size, structure, and specificity.

Related resources

- [Skills](#): package repeatable workflows that load on demand
- [Settings](#): configure Claude Code behavior with settings files
- [Manage sessions](#): manage context, resume conversations, and run parallel sessions
- [Subagent memory](#): let subagents maintain their own auto memory